Day 4 — Security and Data Privacy for GenAI

Saturday 24 Oct 2026 · AWS AIP-C01 exam task covered: **3.2 Implement data security and privacy controls** · Exam: Mon 9 Nov, 1:30 PM IST

Today in one paragraph
Protect data at every point it passes through: source data (Macie, Comprehend), ingestion and the Knowledge Base (KMS, Lake Formation, OpenSearch Serverless policies), the call to the model (PrivateLink, IAM, SCPs), the response (Guardrails PII masking) and the logs (KMS, lifecycle, CloudTrail). Exam method: find **where** the data is in the scenario, then pick the control for that point.

1. The mental model

```
[Source data] -> [Ingestion / KB] -> [Prompt] -> [Model] -> [Response] -> [Logs]
  Macie           KMS, Lake Formation  Guardrails  PrivateLink  Guardrails    KMS, lifecycle
  Comprehend      OpenSearch policies  IAM         IAM / SCP    (PII mask)    CloudTrail
```

2. What Bedrock does with your data

- Prompts and responses are **not used to train** AWS or third-party models and are **not shared with model providers**.
- Data stays **in the region you call**, unless you use cross-region inference. For residency, use a **geographic** inference profile (e.g. US, EU), not a global one.
- Bedrock **doesn't store prompts or outputs** by default; they are kept only if you enable **model invocation logging**.

3. Network isolation — VPC endpoints (PrivateLink)

Requirement	Control
No traffic to Bedrock over the public internet	**Interface VPC endpoint**: **bedrock-runtime** (inference), **bedrock-agent-runtime** (KB Retrieve, agents), **bedrock** (control plane). Enable private DNS.
Only allow Bedrock calls arriving through our endpoint	IAM / resource policy **Deny** with condition **aws:SourceVpce** StringNotEquals the endpoint ID
Limit what can be called through the endpoint	**VPC endpoint policy** (allowed actions, model ARNs)
Lambda tool in a private subnet calls Bedrock	Lambda in the VPC + interface endpoint — **no NAT gateway**
Fine-tuning job reaches training data privately	**VPC configuration on the model customisation job**

Trap
Retrieval and agents use **bedrock-agent-runtime**, not bedrock-runtime.

4. Who can use which model — IAM and SCPs

Requirement	Control
Developers may call only specific models	Identity policy allowing bedrock:InvokeModel / InvokeModelWithResponseStream on **those model (or inference profile) ARNs only**
Block a model across the whole organisation, admins included	**Service Control Policy (SCP)** deny
App needs credentials	**IAM role** (short-term), never long-term access keys
Tool / API secrets	**Secrets Manager**, or **AgentCore Identity** for OAuth tokens
Users retrieve only their own documents	**KB metadata filtering** from their authenticated identity
Cross-account access	**Resource-based policy** or a cross-account role

An explicit **Deny always wins**. An SCP sets the maximum anyone in a member account can do, even administrators and root (it does not apply to the management account).

5. Encryption — KMS

- **Customer managed KMS keys** when the scenario says 'control, rotate, audit key use, revoke access'.
- Used by: Knowledge Bases and ingestion, agents, **custom / fine-tuned models**, customisation job outputs, S3 data sources, invocation logs, OpenSearch Serverless collections.
- The service role (e.g. the KB role) needs **kms:Decrypt** (and **kms:GenerateDataKey** when writing) in the **key policy** or a grant. 'AccessDenied on KB sync after switching to SSE-KMS' → key policy missing the role.

OpenSearch Serverless policy	Controls
Encryption policy	Which KMS key encrypts the collection
Network policy	Public access, or only through a VPC endpoint
Data access policy	Which roles can read / write which indexes

6. PII — find it, redact it, block it

When	Tool	What it does
Before ingestion: PII sitting in S3	**Amazon Macie**	Discovers and reports sensitive data in buckets
In the data pipeline	**Amazon Comprehend** DetectPiiEntities / redaction	Removes PII before chunking and embedding
At runtime: model input / output	**Bedrock Guardrails** sensitive information filters	**Block** or **mask** PII types, plus custom **regex** (e.g. employee IDs)
Images and scanned documents	Bedrock Data Automation / Textract → Comprehend	Extract text first, then redact

'PII must never be embedded in the KB' → **Macie to detect, Comprehend to redact, before ingestion**. 'Don't show PII in responses' → **Guardrails masking** (defence in depth: use both).

7. Data lake access — AWS Lake Formation

Fine-grained permissions on the Glue Data Catalog at **table, column, row and cell** level. 'The agent may read orders but never the card_number column' → **Lake Formation column-level permissions** — bucket policies can't do that.

8. Retention and deletion

- **S3 lifecycle rules**: transition to cheaper storage, then **expire after N days** (invocation logs, transcripts, training data).
- **S3 Object Lock** (WORM): data must not be changed or deleted for a period — 'keep at least' vs lifecycle's 'delete after'.
- Agent memory retention → **AgentCore Memory event expiry** (Day 3).

9. Logging and audit (preview of Task 3.3)

Need	Tool
Who called which Bedrock API, when	**AWS CloudTrail**
Full prompt and response contents	**Model invocation logging** to S3 and/or CloudWatch Logs
Logs contain sensitive data	Encrypt with **KMS**, restrict access, **lifecycle expiry**

10. Decision rules

1. No public internet → **interface VPC endpoints** (bedrock-runtime; **bedrock-agent-runtime** for KB/agents), enforced with **aws:SourceVpce**.
2. Restrict models for a team → **IAM with model ARNs**; whole organisation → **SCP deny**.
3. Own / rotate / audit keys → **KMS customer managed key**; sync fails after enabling KMS → **key policy**.
4. Find PII in S3 → **Macie**; redact in pipeline → **Comprehend**; mask/block at runtime → **Guardrails**.
5. Column / row restrictions on lake data → **Lake Formation**.
6. Delete after N days → **S3 lifecycle**; must not change → **Object Lock**.
7. Data residency → in-region, or a **geographic** cross-region profile.
8. 'Is our data used for training?' → **No**; not stored or shared unless you enable logging.
9. Who called what → **CloudTrail**; prompt contents → **invocation logging** (encrypted, with expiry).
10. OpenSearch Serverless security → **encryption, network and data access policies**.

11. Lab — console, no model quota needed

Status: steps studied in full; not executed yet (no console access today). Run Parts A and G when console access returns — they are the most exam-relevant.

Part	Steps	What to record
A. IAM (15 min)	Create policy **bedrock-nova-only** allowing InvokeModel on the Nova Lite ARN only; attach to a test role; IAM Policy Simulator: Nova Lite → allowed, an Anthropic model → implicitly denied.	Implicit vs explicit deny; how an SCP does this org-wide.
B. VPC endpoint (15 min)	Create interface endpoint **com.amazonaws.us-east-1.bedrock-runtime**, private DNS on; inspect the endpoint policy; find bedrock-agent-runtime. **Delete immediately** (hourly billing).	Which endpoint Retrieve needs; how aws:SourceVpce forces traffic through it.

Part	Steps	What to record
C. KMS (15 min)	Create key **genai-data**; S3 bucket with default **SSE-KMS** using it; read the key policy.	The two KMS permissions a KB role needs.
D. Macie (15 min)	Upload a **fake** customers.csv (name, email, card 4111111111111111); enable Macie trial; run a one-time job on the bucket.	Findings; Macie vs Comprehend vs Guardrails.
E. Lifecycle (5 min)	Lifecycle rule on prefix logs/: expire after 30 days.	When to use Object Lock instead.
F. Invocation logging (5 min)	Bedrock → Settings → Model invocation logging: review S3 / CloudWatch options; don't save.	Why logs need KMS + lifecycle.
G. Guardrail PII (optional)	Guardrail **pii-guard**: Email → Mask, Credit card → Block, regex ORD-\d{6} → Mask; test with **apply-guardrail** in CloudShell (no model call).	What was masked vs blocked.

```
aws bedrock-runtime apply-guardrail --guardrail-identifier <GUARDRAIL_ID> --guardrail-version DRAFT \
  --source OUTPUT --content '[{"text":{"text":"Mail jane@example.com about ORD-123456, card 411111111111111
```

Cleanup
Delete the VPC endpoint (immediately), Macie job (disable Macie if unused), bucket (empty first), KMS key (schedule deletion, min 7 days), IAM policy and test role, guardrail.

12. Day 4 quiz — your answers graded (4.25 / 5)

#	Question	Your answer	Verdict
1	Private subnets, no internet, no NAT: call models and query a KB; stop developers bypassing it.	Interface endpoints bedrock-runtime and 'bedrock-runtime-agents'; endpoint ID in aws:SourceVpce	¾ Right design. Exact name is **bedrock-agent-runtime**. Enforce with a **Deny** when aws:SourceVpce **StringNotEquals** the endpoint ID; enable private DNS.
2	KB ingestion AccessDenied after switching the bucket to SSE-KMS (role still has s3:GetObject).	We need the key policy	½ Be specific: the **KMS key policy** (or a grant) must allow the **KB service role** **kms:Decrypt** (and GenerateDataKey when writing). The exam option will name the permission.
3	Card numbers must never reach the vector index.	Macie to find PII at ingestion, Comprehend to redact before chunking/embedding	✓ Correct. Then ingest the redacted copy; add Guardrails masking at runtime for defence in depth.
4	No account, admins included, may use a specific model.	SCP deny	✓ Correct. SCPs bind all principals in member accounts, including root (not the management account).
5	Agent may read orders but never card_number.	Lake Formation column-level restriction	✓ Correct (service name: **AWS Lake Formation**).

13. Mistakes and learnings

What happened	Correction
Wrote 'bedrock-runtime-agents'.	Exact endpoint: **bedrock-agent-runtime**. Exam options differ by one word.
'We need the key policy' without the permission.	Name it: KMS key policy grants the role **kms:Decrypt**.
Lab not executed (no console access).	Theory is solid; schedule Parts A and G when access returns.

14. Key takeaways

- Locate the data in the scenario, then pick the control for that point.
- bedrock-runtime vs bedrock-agent-runtime; aws:SourceVpce to enforce private paths.
- IAM restricts a team; SCP restricts the organisation, admins included.
- KMS failures are usually the key policy; OpenSearch Serverless has three policy types.
- Macie finds, Comprehend redacts, Guardrails masks at runtime; Lake Formation for columns and rows.

Next: Day 5 — Models and inference (Tasks 1.2, 2.2, 2.4)
Model selection, inference parameters, Converse API and streaming, on-demand vs provisioned vs batch, cross-region inference, fine-tuning and distillation, quotas, throttling and retries.